

目录

2.1 Google文件系统GFS

2.2 分布式数据处理MapReduce

2.3 分布式锁服务Chubby

2.4 分布式结构化数据表Bigtable

2.5 分布式存储系统Megastore

2.6 大规模分布式系统的监控基础架构Dapper

2.7 海量数据的交互式分析工具Dremel

2.8 内存大数据分析系统PowerDrill

2.9 Google应用程序引擎

2.6 大规模分布式系统的监控 基础架构Dapper

- ▶ 2.6.1 基本设计目标
- 2.6.2 Dapper监控系统简介
- 2.6.3 关键性技术
- 2.6.4 常用Dapper工具
- 2.6.5 Dapper使用经验

用户的平均每一次前台搜索会导致Google的后台发生

1011 次的处理

在我们看来，很简单的一次搜索，实际上涉及了众多Google后台子系统，这些子系统的运行状态都需要进行监控。

2.6 大规模分布式系统的监控基础架构Dapper

● 两个基本要求



监控系统设计
两个基本要求

1. 广泛可部署性 (Ubiquitous Deployment)

设计出的监控系统应当能够对尽可能多的
Google服务进行监控

2. 不间断的监控

Google的服务是全天候的，如果不能对Google 的后台同样进行全天候的监控，很可能会错过某些无法再现的关键性故障

2.6 大规模分布式系统的监控基础架构Dapper

• 三个基本设计目标

低开销

这个是广泛可部署性的必然要求。监控系统的开销越低，对于原系统的影响就越小，系统的开发人员也就越愿意接受这个监控系统。

对应用层透明

监控系统对程序员应当是不可见的。如果监控系统的使用需要程序开发人员对其底层的一些细节进行调整才能正常工作的话，这个监控系统肯定不是一个完善的监控系统。

可扩展性

Google的服务增长速度是惊人的，设计出的系统至少在未来几年里要能够满足Google服务和集群的需求。

2.6 大规模分布式系统的监控 基础架构Dapper

2.6.1 基本设计目标

► 2.6.2 Dapper监控系统简介

2.6.3 关键性技术

2.6.4 常用Dapper工具

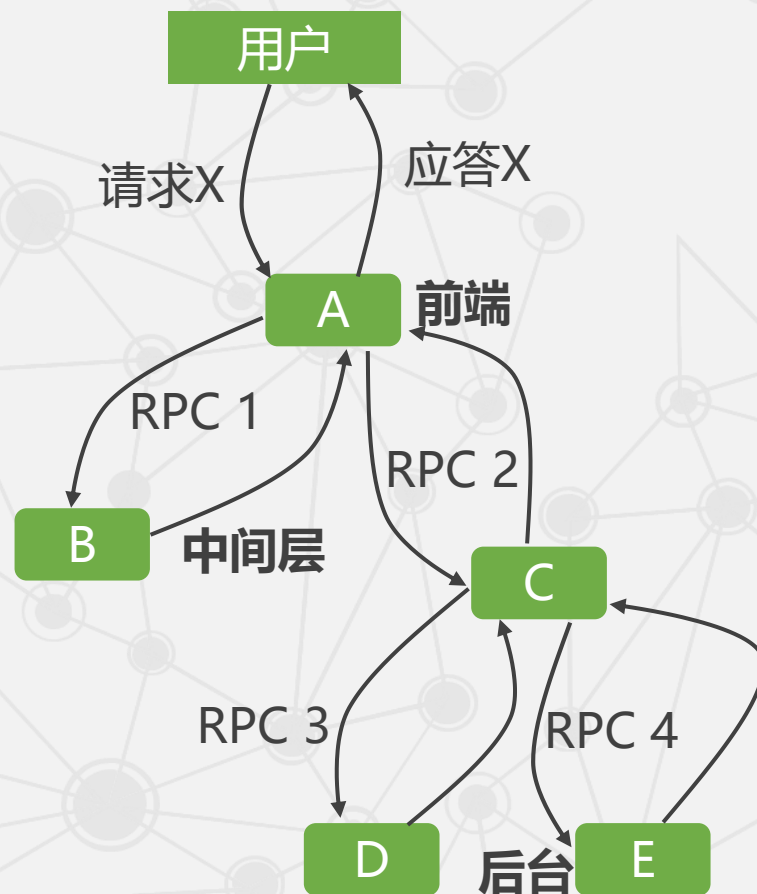
2.6.5 Dapper使用经验

2.6 大规模分布式系统的监控基础架构Dapper

● 1. 基本概念

处理一个用户请求涉及多个子系统和若干消息

- 在监控系统中记录下所有这些消息不难，如何将这些消息记录同特定的请求（本例中的X）**关联**起来才是分布式监控系统设计中需要解决的关键性问题之一。



典型分布式系统的请求及应答过程

2.6 大规模分布式系统的监控基础架构Dapper

● 1. 基本概念

监控系统通常有两种方案：

黑盒 (Black Box)

黑盒方案比较轻便，通过**统计学**来判断消息关系，不够准确。

基于注释的监控 (Annotation-based Monitoring)

利用应用程序或中间件给每条记录赋予一个全局性的**标示符**，借此将相关消息串联起来。

Google选择了**基于注释**的方案。

为了尽可能消除监控系统的应用程序对被监控系统的性能产生的不良影响，Google设计并实现了一套**轻量级**的核心功能库。

2.6 大规模分布式系统的监控基础架构Dapper

• 1. 基本概念

监控树 (Trace Tree)

一个同特定事件相关的所有消息，以**树**的形式组织起来。

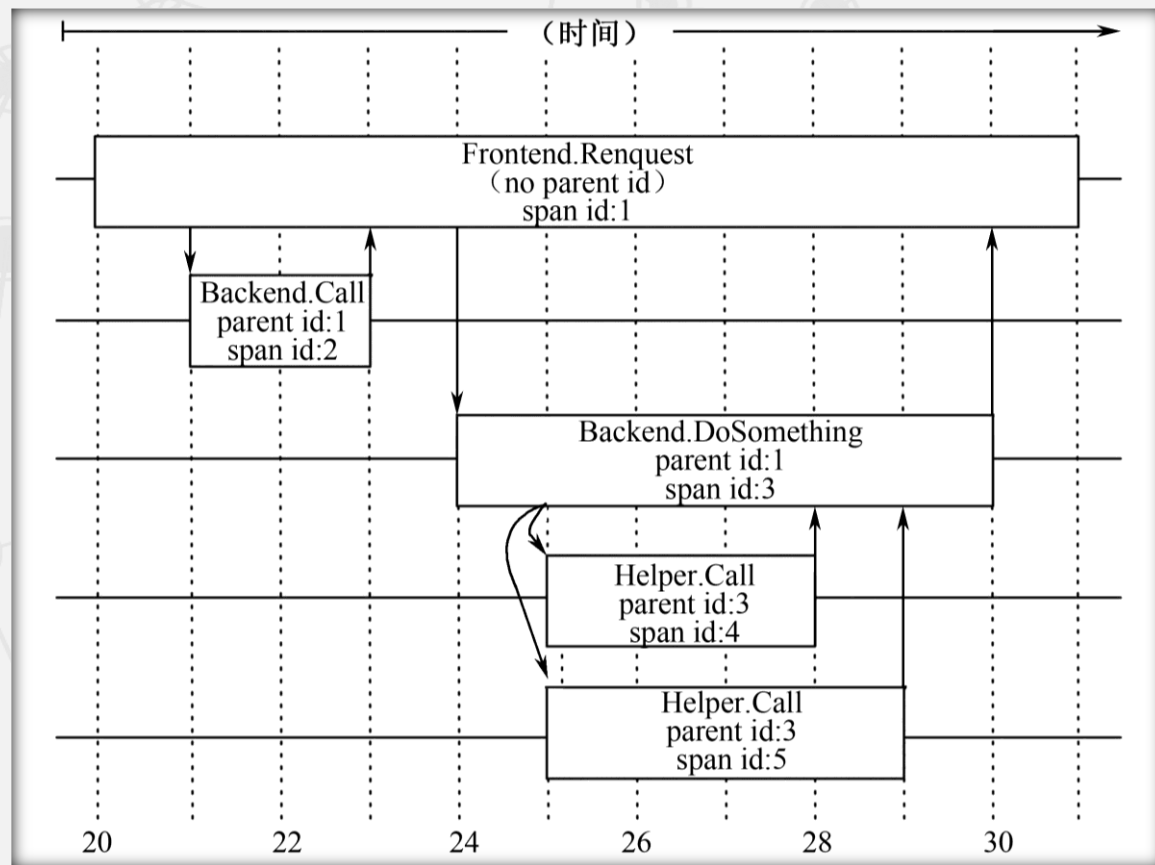
区间 (Span)

树中的每个**节点**就是一个区间，区间实际上就是一条**记录**，所有记录联系起来就是对某个事件的完整记录。

注释 (Annotation)

注释主要用来辅助**推断区间关系**，也可以包含一些自定义的内容。

Dapper监控系统中有三个基本概念：



2.6 大规模分布式系统的监控基础架构Dapper

• 1. 基本概念

区间 (Span)

每个区间包括：

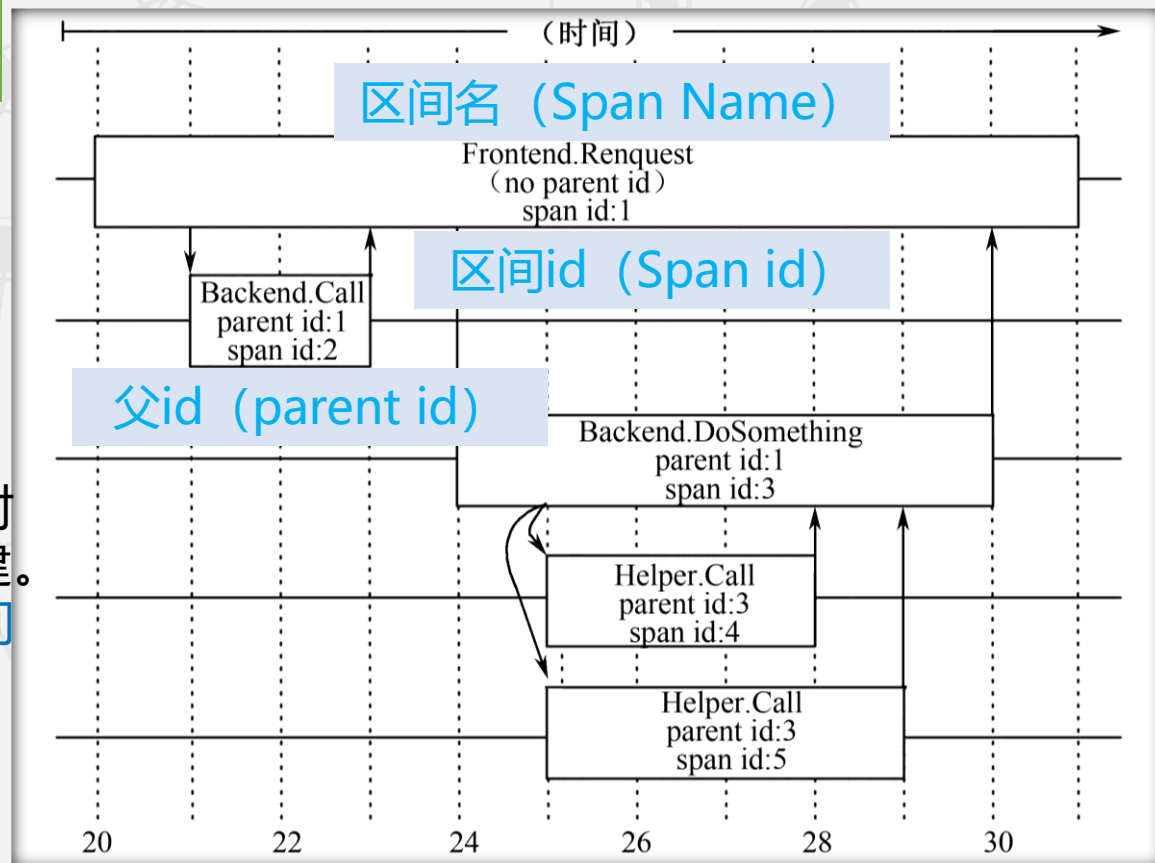
区间名为了方便记忆和理解，要求可以读懂。

区间id为了在一棵树中区分不同的区间。

父id定义了**区间关系**，以便对树中不同区间的关系进行重建。没有父id的区间为**根区间** (Root Span)。

区间的长度包括了区间的开始及结束**时间**信息。

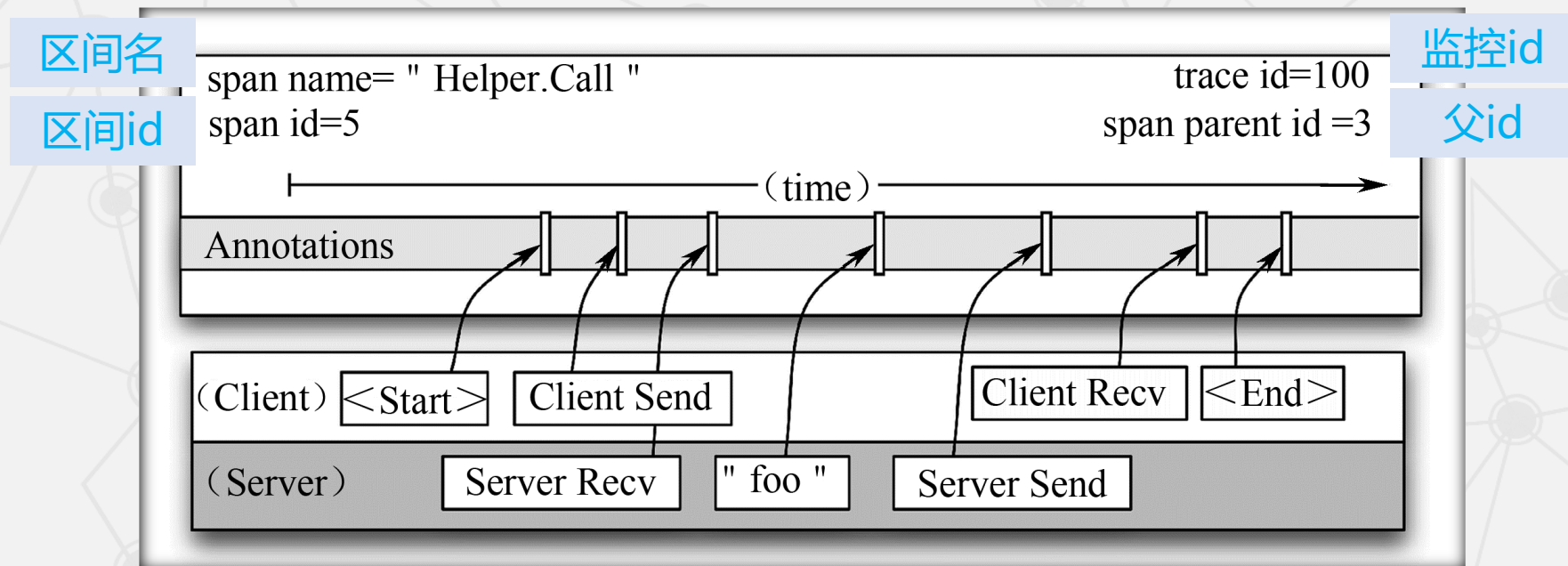
监控id (Trace id)



监控id对每棵树的所有区间是相同的，id内容随机分配，且在**整个**Dapper监控系统中是**唯一**的，用于区分不同的监控。

2.6 大规模分布式系统的监控基础架构Dapper

• 区间Helper.Call的详细信息



双主机区间: 区间包含**客户端**的注释信息: "`<Start>`"、"`Client Send`"、"`Client Recv`" 和 "`<End>`"。也包含**服务器端**的注释信息: "`Server Recv`"、"`foo`" 和 "`Server Send`"。除了 "`foo`" 是用户自定义的, 其他注释都是和时间相关的信息。Dapper除了支持**文本**注释, 还支持**键值对**注释。

2.6 大规模分布式系统的监控基础架构Dapper

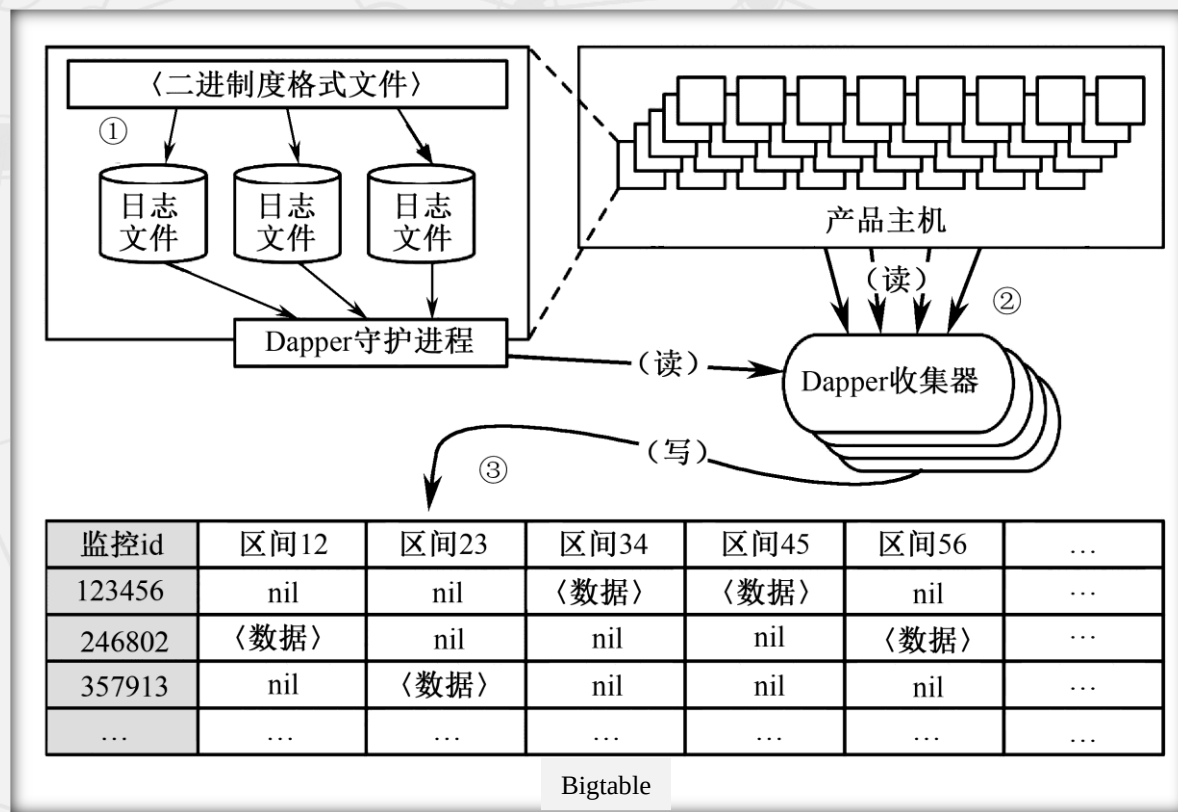
● 2. 监控信息的汇总

海量的消息记录汇集在一起才能产生监控信息。监控信息的汇总需要三步：

(1) 将区间的数据写入到**本地**的日志文件

(2) 所有机器上的**本地**日志文件汇集

(3) 汇集后的数据写入到Bigtable**存储库**中



2.6 大规模分布式系统的监控基础架构Dapper

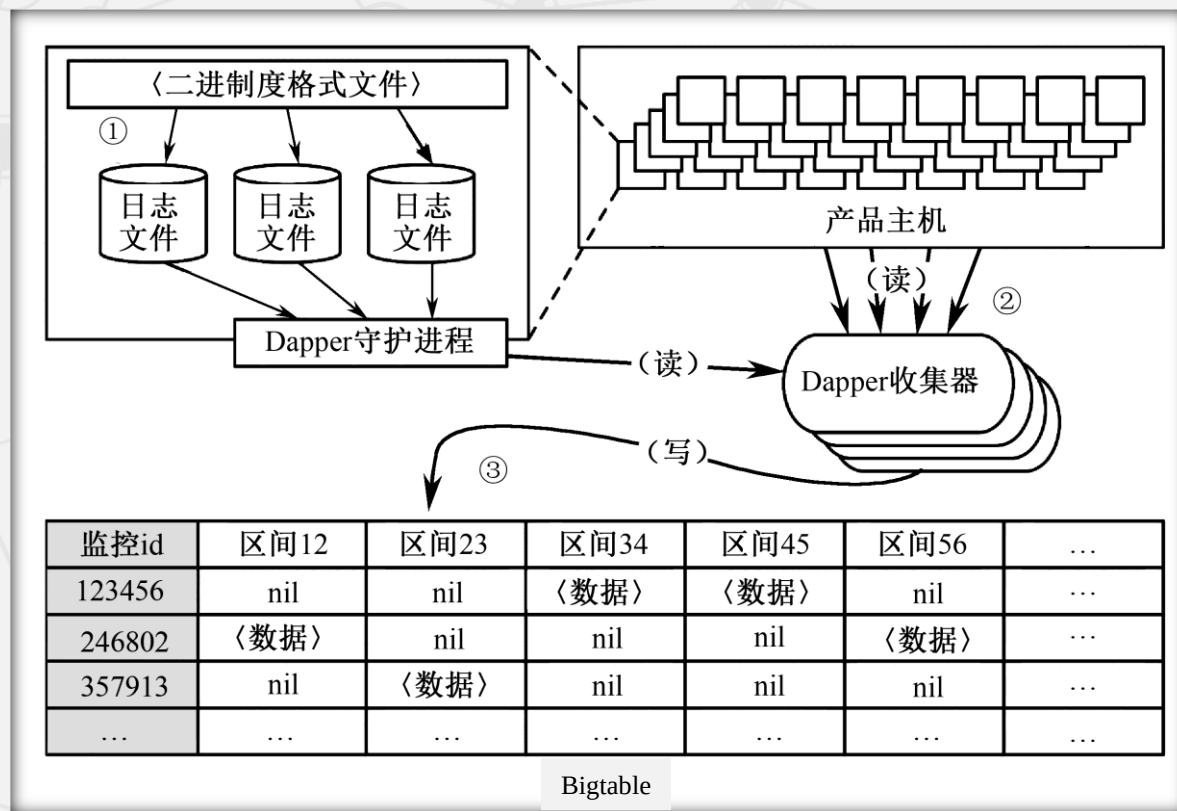
2. 监控信息的汇总

过程 (1) 和 (2) 是读，
(3) 是写。

Why Bigtable?

区间数目众多，且长度不一，而Bigtable能支持松散的表结构。

行：一个记录。
列：不同的区间。



2.6 大规模分布式系统的监控基础架构Dapper

● 2. 监控信息的汇总

监控数据的汇总是单独进行的，即监控数据与应答信息分开传输。原因有二：

(1) 一个内置的汇总方案(监控和应答一起返回)会影响网络动态。一般来说，RPC应答数据规模比较小（<10KB），而区间数据规模非常庞大。如果将监控数据随RPC应答头返回，则会使RPC应答数据相对“矮化”，进而影响后期的

(2) 内置的汇总方案需要保证所有的RPC都是完全嵌套的，但有许多的中间件系统在其所有的后台返回最终结果之前，就对调用者返回结果，导致部分监控信息无法被收集。

2.6 大规模分布式系统的监控基础架构Dapper

● 2. 监控信息的汇总

安全问题是所有系统都必须考虑的，为了防止**未授权**用户对RPC信息的访问，信息汇总过程中Dapper**只**存储RPC方法的**名称**，**不**存储任何RPC**负载数据**。

取而代之的是，**应用层注释**提供了一种方便的**选择性数据开放机制**（Opt-in Mechanism），即应用程序开发者可以将任何对后期分析**有益的数据和区间****关联**起来。

2.6 大规模分布式系统的监控 基础架构Dapper

2.6.1 基本设计目标

2.6.2 Dapper监控系统简介

► 2.6.3 关键性技术

2.6.4 常用Dapper工具

2.6.5 Dapper使用经验

2.6 大规模分布式系统的监控基础架构Dapper

• 2.6.3 关键性技术

低开销

对应用层透明

可扩展性

Dapper有三个设计目标。为了达到此目标，Google采用了两个方面的关键性技术：

1. 轻量级的核心功能库
2. 二次抽样技术

三个目标中，“对应用层透明”是实现难度最大的。

2.6 大规模分布式系统的监控基础架构Dapper

● 1. 轻量级核心功能库——实现对应用层透明

小规模库

通用线程

(Ubiquitous Threading)

控制流

(Control Flow)

RPC代码库

(RPC Library Code)

- Dapper的核心监控是在由通用线程、控制流和RPC代码库组成的**小规模库**上实现的。
- **主要功能**是实现区间创建、抽样和在本地磁盘上记录日志。
- 其核心功能的实现用C++ <1000行代码，Java <800行。键/值对方式注释功能的实现需要额外增加500行代码。
- 将复杂的功能实现限制在一个**轻量级的核心功能库**中保证了Dapper的监控过程基本对应用层透明。

2.6 大规模分布式系统的监控基础架构Dapper

● 2. 二次抽样技术

Google每天需要处理的请求量惊人，如果对所有请求都进行监控，监控数据将会十分庞大，也不利于分析。

监控的开销大小直接决定了Dapper的成败，为了尽可能减小开销，Google设计了二次抽样方案，即两次抽样过程。

第一次 抽样

Dapper对请求进行一次抽样，只有被抽中的请求才会被监控。在实践中，设计人员发现当抽样率低至 $1/1024$ 时也能够产生足够多的有效监控数据，即在1024个请求中抽取1个进行监控也是可行的，已足够捕获有效数据。

这种低抽样率有效的原因在于，巨大的事件数量使关注的事件可能出现的足够多，从而可以捕获有效数据。

最初Dapper设计的是统一的抽样率，但渐渐发现，对于流量较低的服务，低抽样率会导致一些关键性事件被忽略。正设计适应性抽样率方案

2.6 大规模分布式系统的监控基础架构Dapper

● 2. 二次抽样技术

尽管采取抽样监控，产生的监控数据量仍然是惊人的。Dapper每天的监控数据超过1T，如果全部写入Bigtable，效率较低，且存储量也有限，必须定期处理，较少的数据能够保存更长的时间。因此，有了二次抽样。

第二次 抽样

发生在数据写入Bigtable前，具体方法是：将监控id散列成一个标量 z ($0 \leq z \leq 1$)，如果某个区间的 z 小于事先定义好的汇总抽样系数，则保留这个区间并将它写入Bigtable，否则丢弃。

即，在采样决策中，利用 z 值来决定某个监控树是整棵予以保留，还是整棵弃用。只需改动 z 值，就可以改变整个系统的写入数据量。

第一次抽象决定是否监控，第二次抽样决定是否写入bigtable

利用二次抽样技术成功地解决了低开销及广泛可部署性的问题。

2.6 大规模分布式系统的监控 基础架构Dapper

2.6.1 基本设计目标

2.6.2 Dapper监控系统简介

2.6.3 关键性技术

► 2.6.4 常用Dapper工具

2.6.5 Dapper使用经验

常用Dapper工具

Dapper存储API

Dapper用户界面

2.6 大规模分布式系统的监控基础架构Dapper

• 1. Dapper存储API

Dapper的“存储API”简称为 **DAPI**，提供了对分散在区域Dapper存储库 (DEPOTS) 的监控记录的直接访问。一般有以下三种方式访问这些记录。

(1) 通过监控id访问 (Access by Trace id)

利用**全局唯一**的监控id直接访问所需的监控数据。

(2) 块访问 (Bulk Access)

借助MapReduce对数以十亿计的Dapper监控数据的**并行访问**。

(3) 索引访问 (Indexed Access)

Dapper存储库支持**单索引** (Single Index)，将监控id的分配设计成**伪随机的** (嵌入了服务或主机信息)，这是快速访问同“特定服务或主机”相关监控的最好方式。

2.6 大规模分布式系统的监控基础架构Dapper

● 2. Dapper用户界面

大部分的用户在使用Dapper时，都是通过基于Web的[交互式用户界面](#)，下面我们来看其一般性的使用流程。

2.6 大规模分布式系统的监控基础架构Dapper

• 2. Dapper用户界面

(1) 用户需要选择监控对象，包括监控的起止时间、区分监控的模式、衡量开销的标准

Job Selection ?

Start Date: 05/06/2008
Start Hour: 09 ?
End Date: 05/06/2008
End Hour: 10 ?
Cluster: clusterABC
User: user123
Job: jobXYZ

Node Information ?

☐ User
☒ RPC or Span Name
☐ Job
☐ Cluster

Cost Metric ?

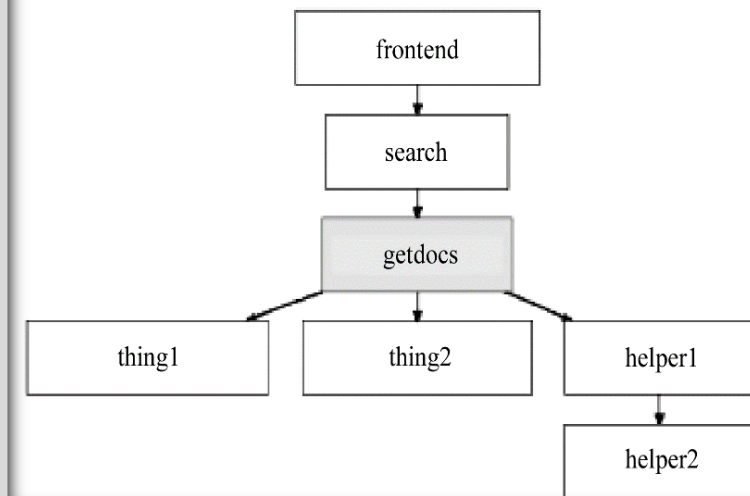
☒ Latency ?
☐ Parent Latency ?
☐ Request Size ?
☐ Response Size ?
☐ Recursive Size ?
☐ Recursive Queue Time ?

(2) 与监控对象有关的分布式执行模式的简要情况如下图所示。用户对这些执行模式进行排序并选择查看更多细节。

Id ?	Calls ?	Total (ms) ?	Global 90%ile Contribution (count) ?	Local 90%ile (ms) ?	Absolute Histogram (ms) ?	Scaled Histogram (ms) ?	View ?
All ?	40,990,720 (100.00%)	139,773,132.8 (100.00%)	4,098,118 (100.00%)	8.91			View
E	3,450,880 (8.42%)	39,437,312.0 (28.22%)	1,918,437 (46.81%)	19.17			View
R	1,658,880 (4.05%)	55,939,686.4 (40.02%)	1,658,880 (40.48%)	47.21			View

(3) 右图是用户所选中的某个分布式执行模式的图形化描述。

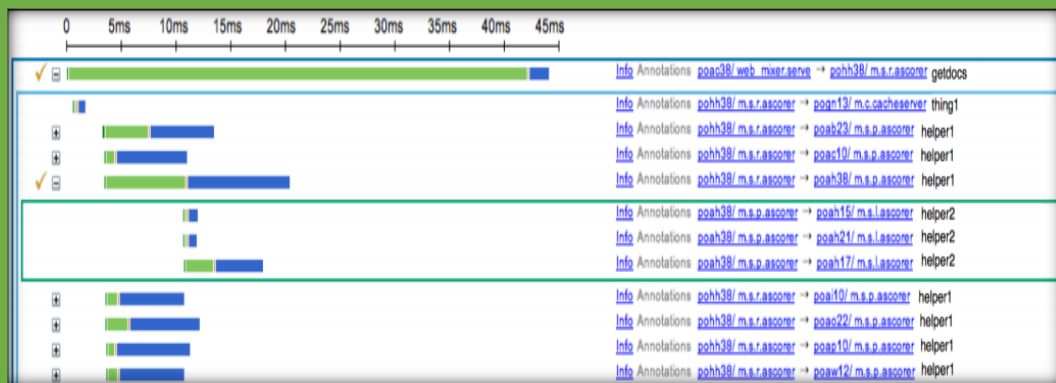
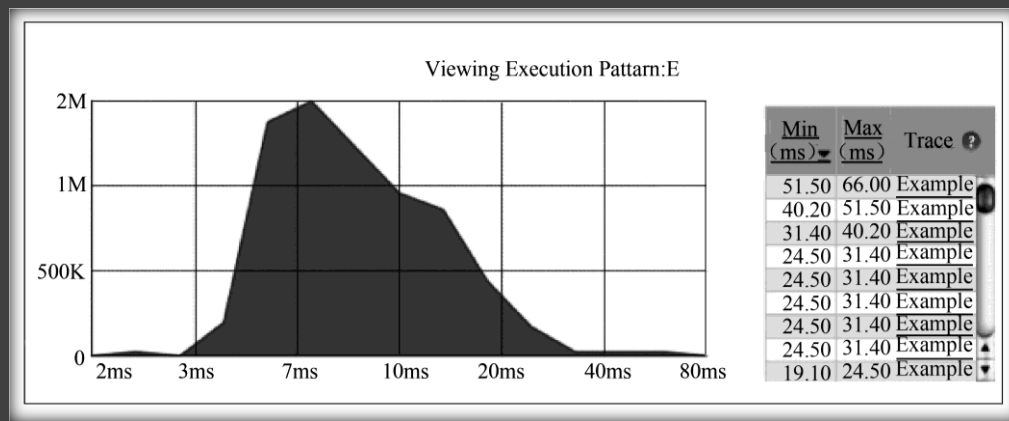
Simplified Call Tree ?



2.6 大规模分布式系统的监控基础架构Dapper

• 2. Dapper用户界面

(4) 根据最初选择的开销度量标准，Dapper以频度直方图的形式将步骤（3）中选中的执行模式的开销分布展示出来。同时还有一些监控样例信息。



(5) 用户选择了某个监控样例后，就会进入监控审查视图。最顶端是全局时间线，每行是一个监控树，可展开或折叠。每个RPC区间分为花在服务器和网络通信上的时间。

2.6 大规模分布式系统的监控基础架构Dapper

据不完全统计，现有3个基于DAPI的持久应用程序，8个额外的基于DAPI的按需分析工具，以及15-20个使用DAPI框架构建的一次性分析工具。

此外，一个普通的工作日内，大概有200个不同的Google工程师在使用Dapper用户界面；一周有750-1000个不同的用户。

2.6 大规模分布式系统的监控 基础架构Dapper

2.6.1 基本设计目标

2.6.2 Dapper监控系统简介

2.6.3 关键性技术

2.6.4 常用Dapper工具

▶ 2.6.5 Dapper使用经验



Dapper在哪些场景中最适用呢？

2.6 大规模分布式系统的监控基础架构Dapper

• Dapper的适用场景

1. 在新服务的部署中
2. 用于定位长尾延迟
3. 用于推断服务间的依存关系
4. 用于确定不同服务的网络使用情况
5. 分层的共享式存储系统
6. 利用Dapper进行“火拼”

- Dapper使用经验

1

新服务部署中Dapper的使用

案例：

Google的AdWords系统在重新开发时，开发团队在从原型系统直到最终版本的发布过程中，反复使用了Dapper对系统的延迟情况进行跟踪，进而发现存在的问题。最终证明，Dapper对于AdWords系统的开发起到了至关重要的作用。

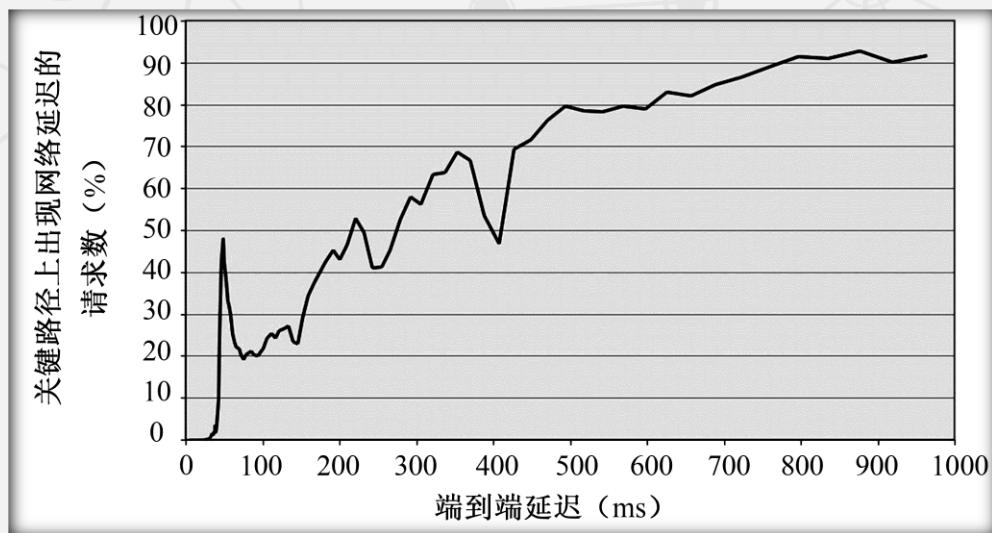
2.6 大规模分布式系统的监控基础架构Dapper

• Dapper使用经验

2

定位长尾延迟 (Addressing Long Tail Latency)

- Google最重要的产品就是搜索引擎，由于规模庞大，对其进行调试时非常复杂的。
- 当用户请求的延迟过长，即延迟时间处于延迟分布的长尾时，即使最有经验的工程师对这种端到端性能表现不好的根本原因也常常判断错误。



- 从图中可以看出，端到端性能和关键路径上的网络延迟有着极大的关系。
- 可以利用Dapper准确地监控到关键路径的延迟。

2.6 大规模分布式系统的监控基础架构Dapper

• Dapper使用经验

3

推断服务间的依存关系 (Inferring Service Dependencies)

- Google的后台服务之间经常需要互相调用，当出现问题时，需要确定该时刻哪些服务时相互依存的，因为这样有利于发现导致问题的真正原因。
- Google的“服务依存关系”项目使用监控注释和DAPI的MapReduce接口实现了服务依存关系确定的自动化。

4

确定不同服务的网络使用情况

- 在Dapper出现之前，Google的网管人员在网络出现故障时，几乎没有工具能够确定到底是哪部分的网络出现的故障。
- 而现在，Google利用Dapper平台构建了一个连续不断更新的控制台，用来显示内部集群网络通信中最活跃的应用层终端。
- 这样，在出现问题时，就可以快速地定位占用网络资源最多的几个服务。

2.6 大规模分布式系统的监控基础架构Dapper

• Dapper使用经验

5

分层的共享式存储系统

- Google中的许多系统都是由多个相对**独立**、且有**复杂层次**的分布式基础架构组成。
- 例如，Google App Engine是构建在一个可扩展的**实体存储系统**之上的。而该实体存储系统则是构建在底层的**Bigtable**之上，展现出一些RDBMS（关系型数据库管理系统）的功能。而Bigtable又依次用到了**Chubby**和**GFS**。
- 在这样的**层次式系统**中，决定终端用户的资源消耗模式并不总是那么简单。
- 例如，由Bigtable的单元引起的GFS高流量，可能主要有一个用户或**几个**用户产生，但在GFS的层次之上，这两种不同的使用模式是**没法分开**的。
- 更进一步，在没有Dapper前，对于这种共享式服务资源的**争用**也同样难以调试。

• Dapper使用经验

6

利用Dapper进行“火拼” (Firefighting with Dapper)

- “火拼”是指处于危险状态的分布式系统的代表性活动。
- 就像消防员 (Firefighter) 在灭火 (Firefighting) 时, 迅速定位火源, 然后采取相应的措施。
- 在分布式系统中, 当系统出现故障或遇到高延迟等问题时, 可以通过使用Dapper来快速定位问题, 并采取紧急措施, 例如, 优化查询语句, 或调整缓存策略等。
- 正在“火拼”中的Dapper用户需要访问最新的数据, 却没有时间来编写新的DAPI代码, 或者等待周期性的报告。
- 此时, 用户可以通过和Dapper守护进程直接通信, 将所需的最新数据汇总在一起。

The background of the image is a dark gray field filled with a complex network of thin, light gray lines. These lines connect numerous circular nodes of varying sizes, creating a web-like or molecular structure. The nodes are also light gray and some have a slight glow or shadow.

本章未完待续